

Kapsamlı Portfolyo Web Uygulaması - Case Çalışması

Merhaba,

Bu case çalışmasında, baştan sona tek katmanlı (Monolithic) bir **ASP.NET Core 8.0 MVC** projesi geliştireceksiniz. Uygulamamız, kişisel bir portfolyo web sitesi ve bu siteyi yönetmek için kullanılacak bir Admin panelinden oluşmaktadır.

Aşağıda projenin teknik isterleri, mevcut mimarisi ve sizden tamamlamanız beklenen görevler detaylandırılmıştır.

Teknik Altyapı ve Kısıtlamalar

- **Teknoloji:** ASP.NET Core 8.0 (MVC)
- **ORM:** Entity Framework Core (Code-First yaklaşımı kullanılacak)
- **Veritabanı:** MS SQL Server
- **Mimari:** Tek katmanlı (Single Layer) yapı.
- **Arayüz (Front-End):** Ön yüz tek bir sayfadan (Single-Page Layout) oluşacaktır. Sayfa içindeki tüm bölümler modüler olarak **ViewComponent**'lar ile beslenecektir.

Mevcut Proje Yapısı (Referans)

Projenin temel omurgası (Entity'ler ve Component isimlendirmeleri) belirlenmiştir. Geliştirmenizi bu isimlendirmelere sadık kalarak yapmanız beklenmektedir.

- **Entities (Varlıklar):** About, Admin, Banner, ContactInfo, Education, Experience, Project, ProjectTechStack, Service, Skill, TechStack, Testimonial, UserMessage.
- **Ön Yüz (Default-Index) Component'ları:** _DefaultAboutViewComponent, _DefaultContactInfoViewComponent, _DefaultUserMessageViewComponent vb.
- **Admin Paneli Component'ları:** _AdminLayoutHeaderViewComponent, _AdminLayoutSidebarViewComponent vb.

Tamamlamanız Beklenen Görevler

Görev 1: Veritabanı ve Base Kurulum

1. Yukarıda listelenen Entity'leri oluşturup DbContext ayarlarınızı yapın.
2. EF Core Migrations kullanarak veritabanınızı SQL Server üzerinde ayağa kaldırın.
3. Ön yüz (UI) tek bir sayfadan oluşmalıdır. Her bir bölüm (Hakkımda, Deneyimler, Projeler vb.) ilgili **ViewComponent** üzerinden veritabanından dinamik olarak çekilerek tek bir Layout üzerinde sergilenmelidir.

Görev 2: Admin Paneli ve Kimlik Doğrulama (Login)

1. Projede bir Admin paneli bulunmalıdır ve bu panele sadece yetkili kullanıcılar erişebilmelidir.
2. **Özel Login Tasarımı:** Hazır bir template kullanmak yerine, **kendi login ekranı tasarımınızı (UI) sıfırdan oluşturmanız (stitch ile AI kullanarak oluşturmanız)** beklenmektedir.
3. Başarılı giriş yapan kullanıcı, Admin dashboard'una yönlendirilmelidir.

4. Dashboard tasarımı içinde istatistik bilgileri tutulmalıdır. Örnek: Gelen Mesaj Sayısı, toplam deneyim sayısı, toplam geliştirilen proje sayısı vs. ve hızlı bakış tablosu yer alabilir. Örnek: Son yapılan projeler..
5. Tüm entityler için CRUD işlemleri yapılacak sayfalar tamamlanacak ve sidebardan yönlendirme yapılacak ve ilgili alanla ilgili ikonlar seçilecek.

Görev 3: İletişim Formu ve Mesaj Yönetim Sistemi

Sitenin ön yüzündeki iletişim formundan (UserMessage) gelen mesajların Admin panelinde yönetilmesi için spesifik bir akış tasarlamamız gerekmektedir:

1. Gelen mesajlar Admin panelinde bir liste halinde görüntülenmelidir.
2. **Otomatik Okundu Bilgisi:** Admin, gelen bir mesajın detayına tıkladığında (mesajı açtığında), veritabanında o mesajın durumu **otomatik olarak "Okundu" (Read) olarak güncellenmelidir.**
3. **Filtreleme/Ayrıştırma:** Gelen Kutusu ekranında okunan ve okunmayan mesajlar birbiriyle karışmamalıdır. Bu ayrımı yapmak için "Okunanlar" ve "Okunmayanlar" şeklinde **iki ayrı buton** veya sayfa içi **filtreleme (Filter)** mekanizması yapmanız zorunludur.

Değerlendirme Kriterleri

Projeniz incelenirken aşağıdaki noktalara dikkat edilecektir:

- **Klasör ve İsimlendirme Standartları:** Görselde verilen ViewComponents klasör yapısına ve isimlendirme kurallarına uyulması.
- **UI/UX Bütünlüğü:** Login sayfasının özgünlüğü ve Admin panelinin derli toplu çalışması.
- **İş Mantığı (Business Logic):** Mesajların detayına girildiği an statüsünün değişmesi ve filtreleme mekanizmasının hatasız (bug-free) çalışması.
- **Clean Code:** Controller'ların şişirilmemesi, veritabanı sorgularının doğru ve performanslı atılması.

Başarılar dilerim! Kolay gelsin.